Royal University of Bhutan

འབྲུག་རྒྱལ་འཛིན་གཙུག་ལག་སློབ་སྡེ།

SAMTSE COLLEGE OF EDUCATION

Title of the Assignment: Exploring GUI Frameworks

Name: Dorji Wangmo

Student Number:08230124

Programme: B.Ed ICT “D”

Samtse College of Education

Samtse

Royal University of Bhutan

Module Code and Title: PRG304

Module Tutor: Anup Gurung

Date:8/11/2026

1. Abstract

The Personal Expense Tracker is a desktop-based application developed using Python and the PyQt6 graphical user interface (GUI) framework. The application is intended to help users record and monitor their personal income and expenses in one place. It provides a structured interface for entering financial transactions, reviewing a financial summary, examining monthly information and setting budget limits for spending categories.

The application uses SQLite as its database so that transaction and budget information can be stored locally and remain available after the program is closed. The interface is organized into different sections, including Dashboard, Transactions, Reports & Analytics, Budget Planner and About. The Dashboard presents an overview of income, expenses and balance, while the Transactions section supports the management of financial records. Searching and month filtering help users locate relevant records. Reports & Analytics presents monthly and category-based information, and the Budget Planner provides a place to manage category budget limits.

The assignment also involves exploring Python GUI frameworks through small Hello applications. PyQt6, Kivy and Tkinter are considered in this report. Their main characteristics and approaches to interface development are compared before explaining the selection of PyQt6 for the main application. The development process provided practical experience in GUI widgets, layouts, event handling, database integration, input validation, modular programming and software testing.

2. Introduction

2.1 Background

Managing personal finances involves keeping track of money received and money spent. When records are kept in notebooks or scattered across separate files, it may take additional effort to calculate totals, review spending patterns and remember previous transactions. A simple digital system can make these activities more organized by storing records in a consistent format and presenting useful summaries.

A graphical user interface makes software easier to operate by allowing users to interact through visible controls such as buttons, text fields, tables and menus. Python provides several GUI frameworks that can be used to build desktop applications. For this assignment, PyQt6 was used to develop a Personal Expense Tracker, while Kivy and Tkinter were explored through smaller demonstration programs.

2.2 Problem Statement

A person who records income and expenses manually may find it difficult to maintain an up-to-date view of their financial position. Calculating monthly totals, locating a particular payment and comparing spending against a budget can become time-consuming when information is not stored centrally. Mistakes may also occur when amounts or descriptions are entered inconsistently.

The Personal Expense Tracker addresses these issues by providing a single desktop application for recording income and expenses, viewing summary information, searching and filtering transactions,

and managing category budget limits. It is designed as a basic personal finance tool rather than a full banking or accounting system.

2.3 Aim and Objectives

The main aim is to develop a desktop application that helps users record and review personal financial transactions using a graphical interface and a local SQLite database.

- To explore the basic operation of PyQt6, Kivy and Tkinter through Hello World programs.
- To design a clear desktop interface using PyQt6 widgets and layouts.
- To provide functions for adding, editing, deleting, searching and filtering transactions.
- To display income, expenses and balance information on a dashboard.
- To provide monthly and category-based financial summaries.
- To allow users to set and view budget limits for spending categories.
- To store application records in SQLite so that data persists between sessions.
- To test important application operations and input validation.

2.4 Scope of the Project

The project focuses on core personal finance functions. The known scope includes a dashboard, transaction management, reports and analytics, a budget planner and an About section. Users can add income and expense records, edit or delete records, search by description or category, and filter records by month. The dashboard displays total income, total expenses and the remaining balance. The budget planner supports category limits.

The application is intended for local personal use. It should not be described as a banking application, tax system or multi-user cloud service unless such functions are added and verified. Features such as online bank synchronization, automatic receipt scanning and cloud backup are outside the confirmed scope of this draft.

2.5 Target Users

The intended users are individuals who want a straightforward way to record their personal income and expenses. The interface is suitable for users who prefer working with forms, buttons and tables instead of maintaining financial records manually. The application may be useful to students or other individuals who want to review their monthly spending and set basic category budgets.

3. Framework Study

This section discusses the three GUI frameworks explored for the assignment: PyQt6, Kivy and Tkinter. The framework comparison is based on their general characteristics.

3.1 Framework Profiles

3.1.1 PyQt6

PyQt6 is a set of Python bindings for the Qt 6 application framework. It allows Python developers to create graphical applications using Qt widgets and other interface components. Common controls include buttons, labels, text fields, tables, menus and dialog windows. Qt provides layout managers that help arrange widgets and resize them when the application window changes.

PyQt6 commonly uses the signals-and-slots mechanism for event handling. A signal is emitted when an event occurs, such as a button being clicked, and it can be connected to a slot or function that responds to that event. Interfaces can be created programmatically or with visual design tools such as Qt Designer. PyQt6 is suitable for feature-rich desktop applications on supported desktop operating systems.

Relevance to the project: PyQt6 was selected for the Personal Expense Tracker because it provides a broad collection of widgets, structured layout management and event-handling facilities suitable for a multi-section desktop application.

3.1.2 Kivy

Kivy is an open-source Python framework for developing multi-touch applications. It can be used on desktop platforms and mobile operating systems. Its controls include labels, buttons, text inputs and layout containers. Kivy also provides a language called KV that can be used to describe interface structure separately from Python application logic.

Kivy supports event handling through event bindings and properties. Its layout options include containers such as BoxLayout and GridLayout. It is often considered when an application needs touch interaction or cross-platform mobile support. Its interface conventions differ from those of traditional desktop GUI frameworks.

Relevance to the assignment: Kivy was explored through a small Hello message program to understand how it creates an application window, displays controls and responds to a user's interaction.

3.1.3 Tkinter

Tkinter is Python's standard interface to the Tcl/Tk GUI toolkit and is commonly included with standard Python installations. It provides basic widgets such as labels, buttons, entry fields, checkboxes, menus and dialog boxes. Its layout managers include Pack, Grid and Place, which arrange widgets inside a window.

Tkinter responds to user actions through callback functions and event bindings. The mainloop() method starts the event loop, allowing the window to remain active and respond to user input. Tkinter is often used for beginner projects, small desktop utilities and prototypes because its basic structure is relatively straightforward.

Relevance to the assignment: Tkinter was explored through a Hello program containing a greeting and a button that responds to a click.

3.2 Framework Comparison Matrix

Comparison area	PyQt6	Kivy	Tkinter
Main purpose	Feature-rich desktop GUI applications	Touch-oriented and cross-platform interactive applications	Basic desktop GUI applications and prototypes
Underlying toolkit	Qt 6	Python framework using its own graphics	Tcl/Tk

		and input system	
Widgets	Large collection of desktop widgets	Kivy widgets and layouts	Common standard GUI widgets
Layout approach	Qt layout managers	Kivy layout containers and KV language	Pack, Grid and Place
Event handling	Signals and slots	Event bindings and properties	Callbacks and event bindings
Interface styling	Qt stylesheets and widget properties	KV language, canvas and widget properties	Widget options and themed ttk widgets
Typical use	Desktop utilities and complex GUI software	Touch-based and mobile applications	Small tools, learning and prototypes
Learning considerations	Broader API and concepts to learn	Different UI model and touch-oriented concepts	Accessible starting point for basic GUI development

3.3 Hello Program Testing

A Hello message program is a small application used to demonstrate the basic operation of a programming framework. In this assignment, each demonstration should open a window, display a greeting and provide a button. When the button is clicked, the application updates a message within the same window.

3.3.1 PyQt6 Hello message Program

The PyQt6 demonstration creates an application object and a main window. Widgets such as a label and a push button are placed in the window. The button's clicked signal is connected to a function that updates the greeting. The application's event loop is started so that the window can receive and process user actions.

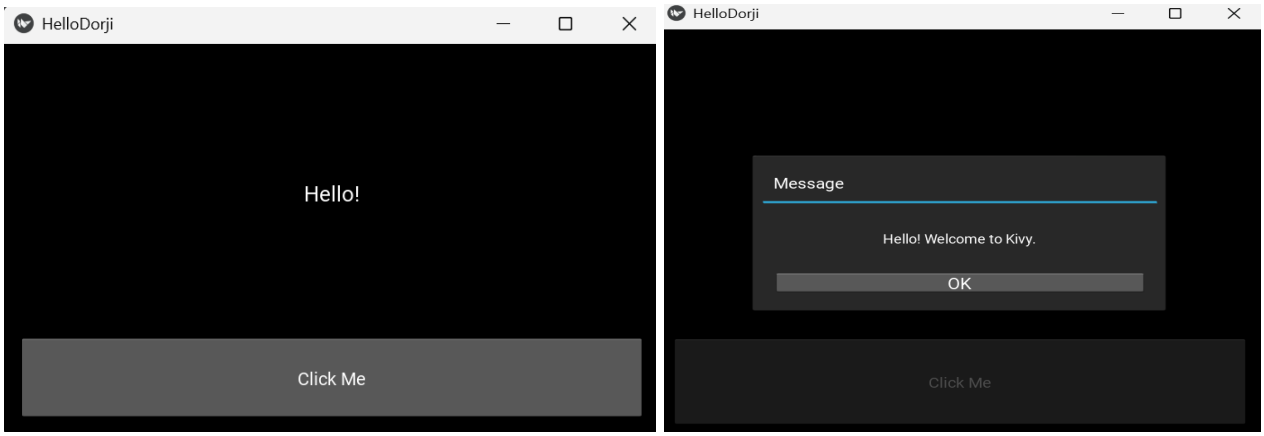


Figure 1: PyQt6 Hello application window.

3.3.2 Kivy Hello Program

The Kivy demonstration defines an application class and builds a widget layout for the greeting and button. A callback is associated with the button so that a click changes the displayed message. The Kivy application runner starts the application and manages its interaction loop.

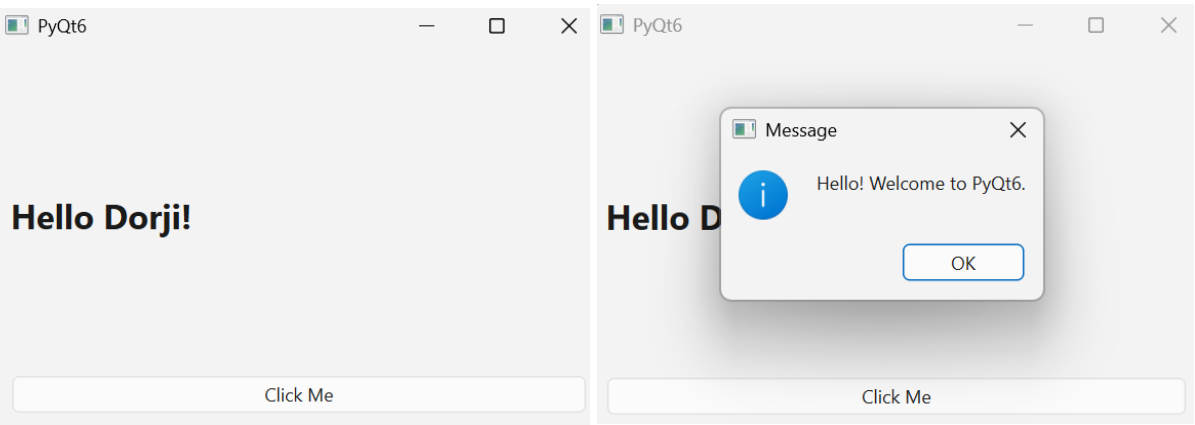


Figure 2: Kivy Hello application window.

3.3.3 Tkinter Hello Program

The Tkinter demonstration creates a root window and adds a label and a button. The button's command option can be connected to a Python function that changes the label text. The mainloop() method keeps the window open and processes events such as button clicks.

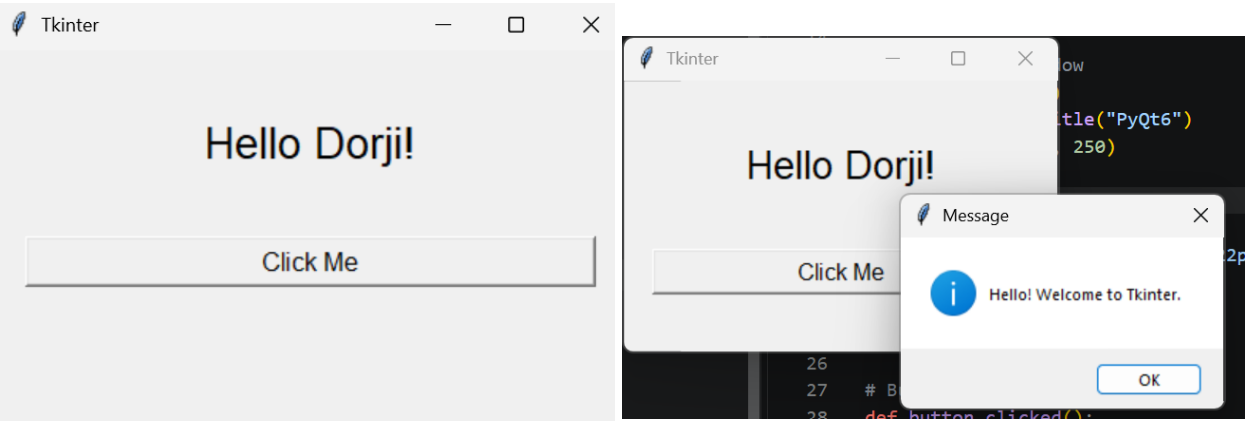


Figure 3: Tkinter Hello World application window.

3.4 Differences Observed During Implementation

The three frameworks provide the same basic result but organize application code differently. PyQt6 uses Qt objects, widgets and signals and slots. Kivy uses its own application and widget model, with layouts and event bindings. Tkinter uses a root window, widget constructors and callback functions. Their layout systems also differ: Qt uses layout classes, Kivy uses layout widgets, and Tkinter offers Pack, Grid and Place.

The Hello message demonstrate that a GUI application needs more than visible controls. It must also create a window, place the controls, connect user actions to functions and start the appropriate event

loop. These common ideas apply to all three frameworks even though their APIs and coding styles are different.

3.5 Recommendation and Justification for Selecting PyQt6

PyQt6 was selected for the main application because the Personal Expense Tracker contains several sections and needs a structured interface for forms, tables, navigation and summary information. Qt's widget collection and layout system support this type of desktop interface. Its signal-and-slot mechanism also provides a clear way to connect buttons and other controls to application functions.

Tkinter would also be suitable for a smaller desktop utility, while Kivy is particularly useful when touch-based or mobile interaction is a priority. For this project, PyQt6 offered a suitable balance of desktop-oriented controls, layout flexibility and interface customization.

4. System Design

4.1 Functional Requirements

ID	Requirement	Description
FR01	Dashboard summary	Display total income, total expenses and remaining balance.
FR02	Add transaction	Allow a user to enter an income or expense record.
FR03	Edit transaction	Allow an existing transaction to be updated.
FR04	Delete transaction	Allow a selected transaction to be removed.
FR05	Search transactions	Search records using description or category.
FR06	Month filtering	Filter displayed transaction records by month.
FR07	Reports and analytics	Present monthly financial summaries and category-based expense information.
FR08	Budget planner	Allow category budget limits to be entered and viewed.
FR09	Input validation	Reject invalid inputs such as a zero transaction amount or empty description, as confirmed by testing.
FR10	Persistent storage	Use SQLite so saved information remains available after the application is closed.

4.2 Non-Functional Requirements

Requirement	Description
Usability	The interface should make common actions understandable through visible controls and organized sections.
Reliability	The application should handle valid and invalid input appropriately and avoid unintended data changes.
Data persistence	Saved records should remain available after closing and reopening the application.
Maintainability	Separating interface, logic and database responsibilities can make the code easier to understand and update.
Responsiveness	The application should respond to normal user actions without unnecessary delay.

4.3 User Interface Design

The application is organized into separate sections so that users can move between financial tasks without treating every operation as a separate program. The Dashboard provides an overview, Transactions is used to manage individual records, Reports & Analytics presents summaries, Budget Planner deals with spending limits, and About provides information about the application.

4.3.1 Dashboard

The Dashboard is the starting overview of the application. It displays total income, total expenses and the remaining balance. These summary values help users understand their current financial position without reviewing every transaction individually. The Dashboard also includes a month selector that allows users to view income, expenses and balance information for a selected month.

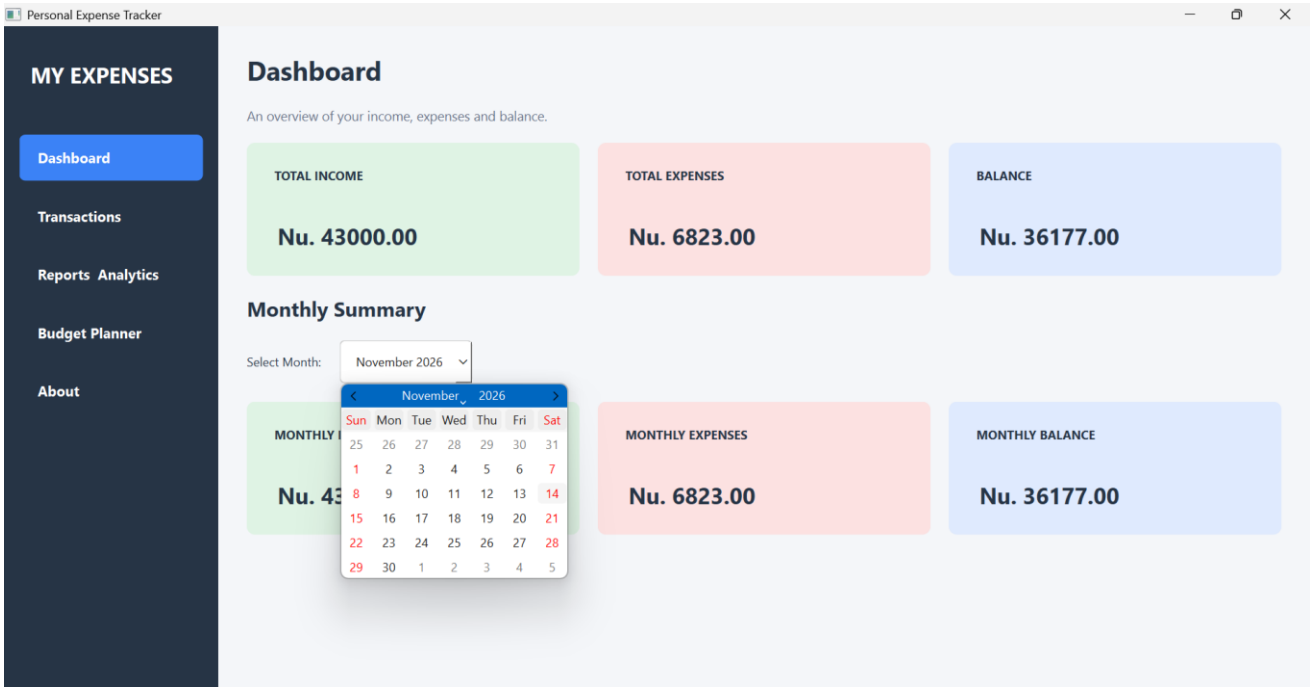


Figure 4: Personal Expense Tracker Dashboard.

4.3.2 Transactions

The Transactions section allows users to manage their income and expense records. Users can add new transactions, edit existing records and delete selected records. The page also includes a search field that finds transactions by description or category. A checkbox allows users to filter records by the selected month. All saved transactions are displayed in a table containing the transaction ID, type, amount, category, date and description.

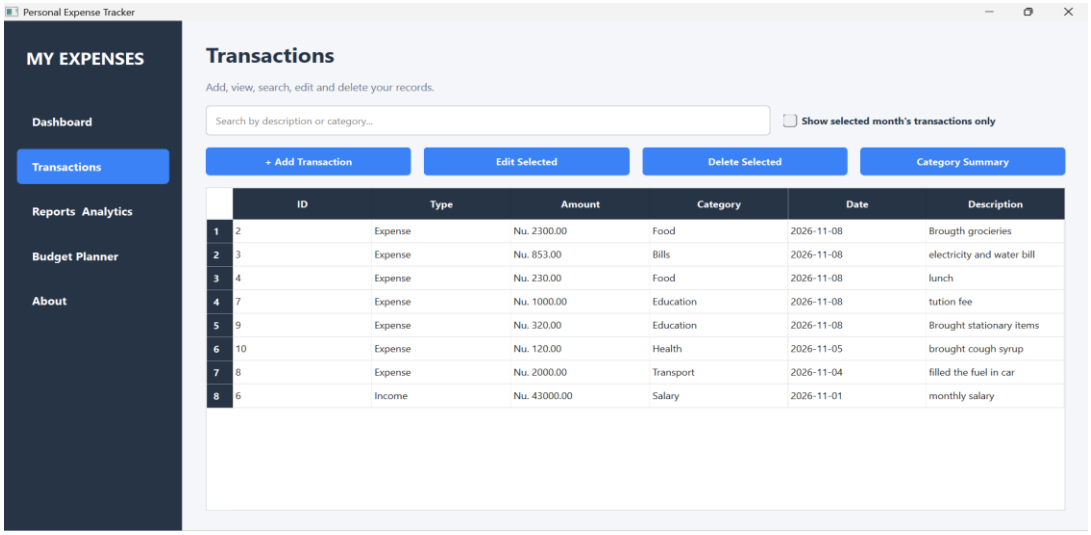


Figure 5: Transactions management page.

4.3.3 Reports & Analytics

The Reports & Analytics section displays financial information for a selected month. It shows monthly income, expenses and balance, along with a comparison using progress bars. It also provides an expense breakdown by category, allowing users to see how much they have spent in each category. A scroll area is included so users can access the full report content.

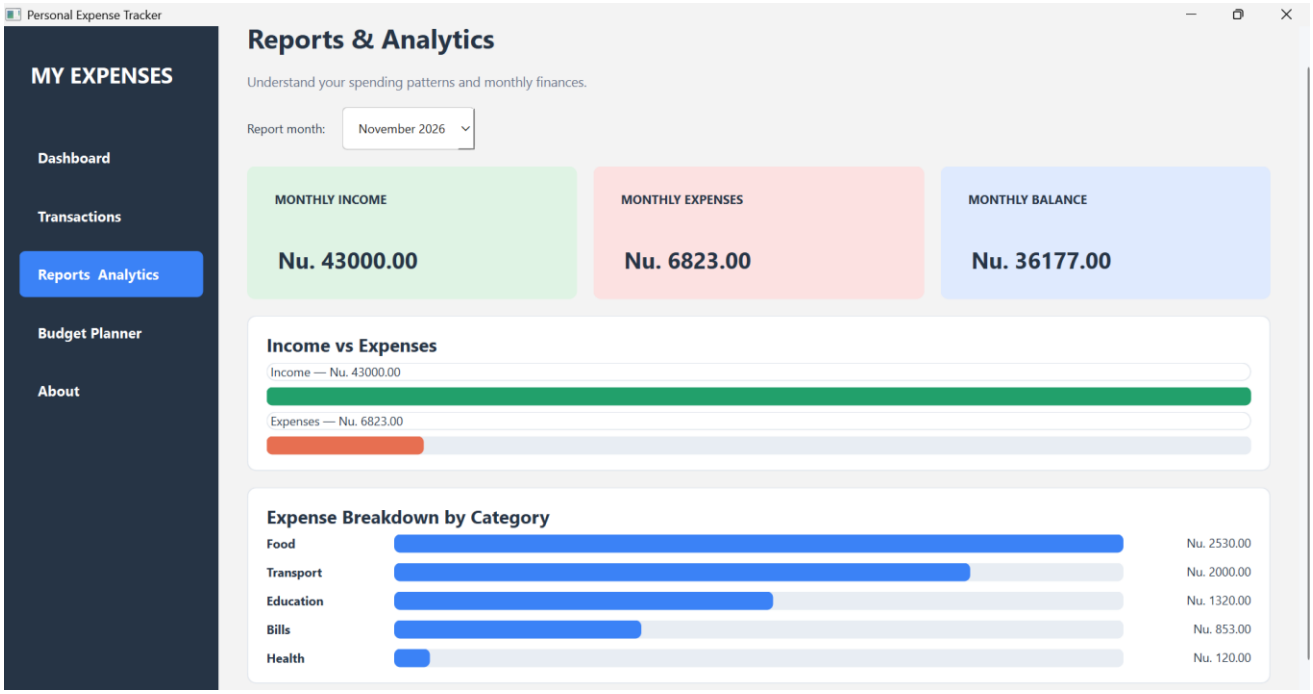


Figure 6: Reports & Analytics page.

4.3.4 Budget Planner

The Budget Planner allows users to create monthly spending limits for different categories, such as Food, Transport, Shopping and Education. Users can save, edit and delete budget records. The page also displays the budget limit, amount spent, remaining amount and percentage of budget used. The spending information is calculated using the saved expense transactions for the selected category and month.

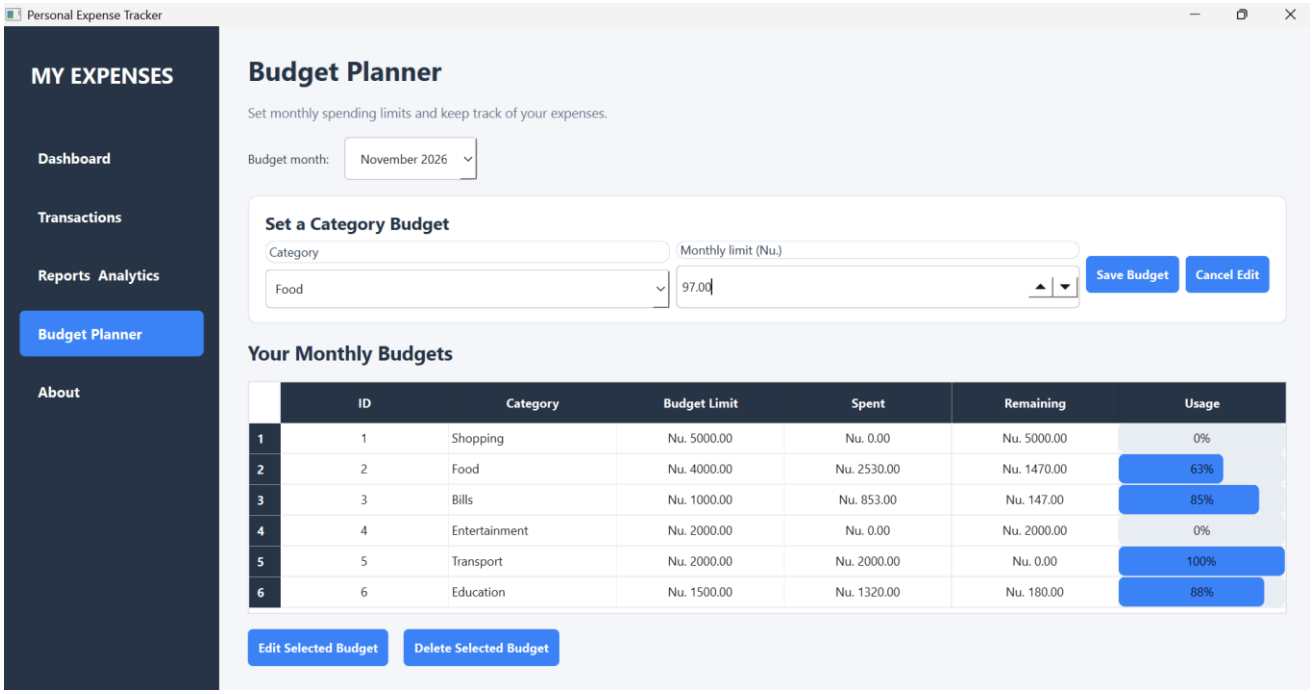


Figure 7: Budget Planner page.

4.3.5 About

The About section provides information about the application and its developer. It helps identify the purpose of the program and provides a place for project or student information.

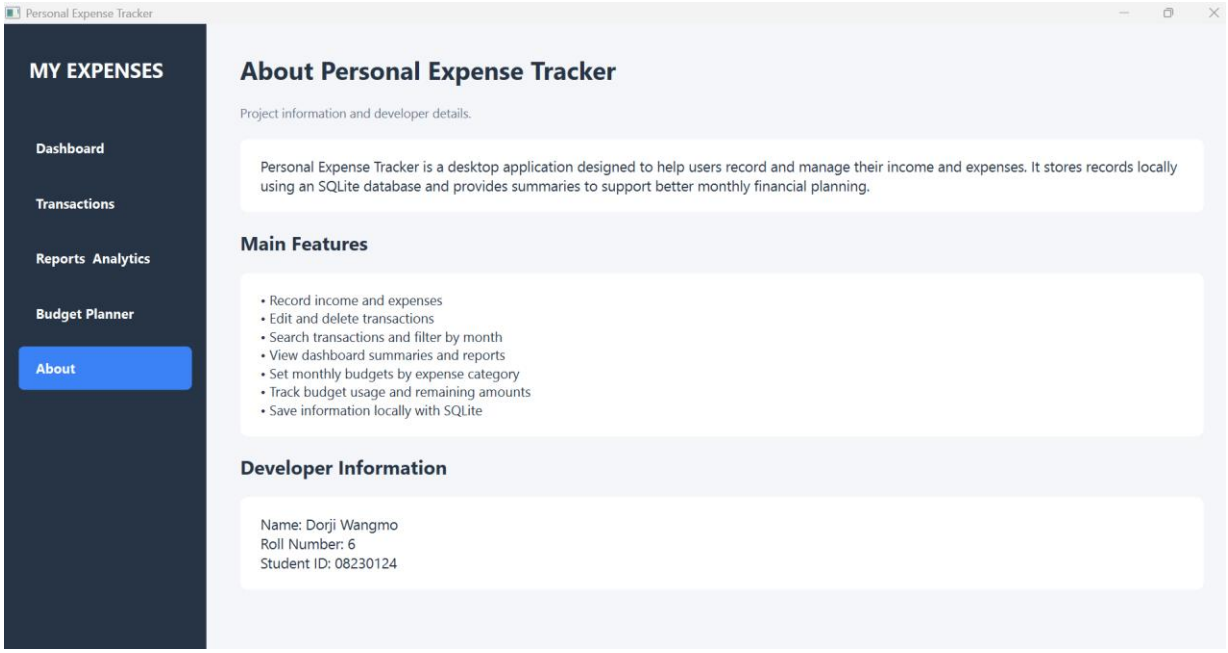


Figure 8: About page.

4.4 Widgets and Layout Design

PyQt6 provides widgets that represent visible interface elements and layout managers that arrange those elements. The following table gives examples of common Qt controls that may be relevant.

Widget / class	Purpose in the application
QMainWindow	Creates the main application window.
QWidget	Acts as a container for different interface sections.
QLabel	Displays headings, descriptions and financial summary values.
QPushButton	Provides actions such as Add, Edit, Delete and navigation.
QLineEdit	Allows users to enter search terms and transaction descriptions.
QComboBox	Allows users to select transaction types and categories.
QDateEdit	Allows users to select dates and months.
QDoubleSpinBox	Allows users to enter numerical budget amounts.
QTableWidget	Displays transaction records and budget information in rows and columns.
QDialog	Provides a separate window for adding and editing transactions.
QMessageBox	Displays warning, confirmation and success messages.
QCheckBox	Allows users to enable or disable the selected-month transaction filter.
QStackedWidget	Holds the different pages and switches between them.
QScrollArea	Allows users to scroll through the Reports & Analytics page.
QProgressBar	Displays income and expense comparisons and budget usage.
QVBoxLayout	Arranges widgets vertically.
QHBoxLayout	Arranges widgets horizontally.
QGridLayout	Arranges summary cards in a grid.
QFormLayout	Arranges transaction input fields with their labels.

4.5 Database Design

SQLite is used as the application's local database. A database allows financial records to be stored in a structured way and retrieved when the application needs to display transactions or calculate summaries. Because the database is stored locally, records can remain available between application sessions.

4.5.1 Database Table Details

Table 1: Transactions

Field name	Data type	Key / Constraint	Purpose
id	INTEGER	Primary Key, AUTOINCREMENT	Uniquely identifies each transaction.
transaction_type	TEXT	NOT NULL	Stores whether the record is Income or Expense.
amount	REAL	NOT NULL	Stores the transaction amount.
category	TEXT	NOT NULL	Stores the transaction category, such as Food or Salary.
date	TEXT	NOT NULL	Stores the transaction date.
description	TEXT	None (nullable)	Stores additional information about the transaction.

Table 2: Budgets

Field name	Data type	Key / Constraint	Purpose
id	INTEGER	Primary Key, AUTOINCREMENT	Uniquely identifies each budget record.
category	TEXT	NOT NULL	Stores the spending category.
month	TEXT	NOT NULL	Stores the month for the budget.
amount	REAL	NOT NULL	Stores the budget limit.
category + month	—	UNIQUE constraint	Prevents duplicate budget entries for the same category and month.

The database contains two tables: transactions and budgets. The transactions table stores the income and expense records entered by the user, while the budgets table stores monthly spending limits for different categories. Both tables use an automatically generated ID as their primary key. The NOT NULL constraint is used for important fields to ensure that required information is provided. The budgets table also has a UNIQUE constraint on category and month to prevent duplicate budget records for the same category in a particular month.

4.5.2 Database Relationships and ER Diagram

An entity-relationship (ER) diagram represents the database tables and the relationships between them. A primary key uniquely identifies a row in a table. A foreign key refers to a key in another

table and can be used to connect related records. Cardinality describes how many records in one table may be associated with records in another table.

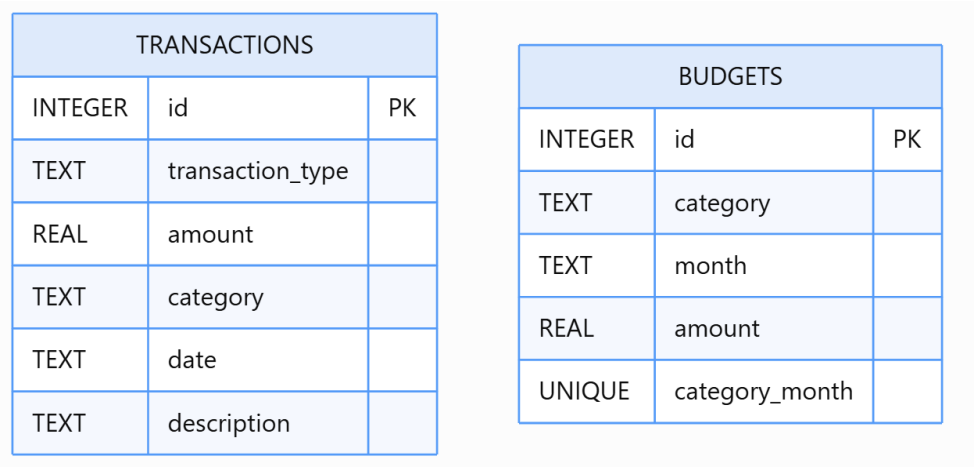


Figure 9: Entity-Relationship Diagram of the Personal Expense Tracker database.

The ER diagram shows the two tables used in the application. The transactions table stores income and expense information, while the budgets table stores monthly budget limits. Each table has its own primary key called id. The budgets table also uses a unique constraint on the combination of category and month. There is no direct foreign key relationship between the two tables in the current database design.

4.6 System Architecture

The Personal Expense Tracker uses a modular architecture that separates the graphical interface, application logic and database operations. The user interacts with the application through the PyQt6 graphical interface, which is developed in main.py. The main window contains five sections: Dashboard, Transactions, Reports & Analytics, Budget Planner and About.

The logic.py module handles calculations, searching and filtering of transaction records. It is used by the main interface to calculate financial summaries and find the required records.

The database.py module handles communication with the SQLite database. It provides functions to add, retrieve, update and delete transaction and budget records. The database information is stored locally in expenses.db.

The expense_dialog.py file provides the form used when adding or editing transactions. After the user enters the required information, the main application sends the data to the database module for storage. The updated information is then displayed in the interface.

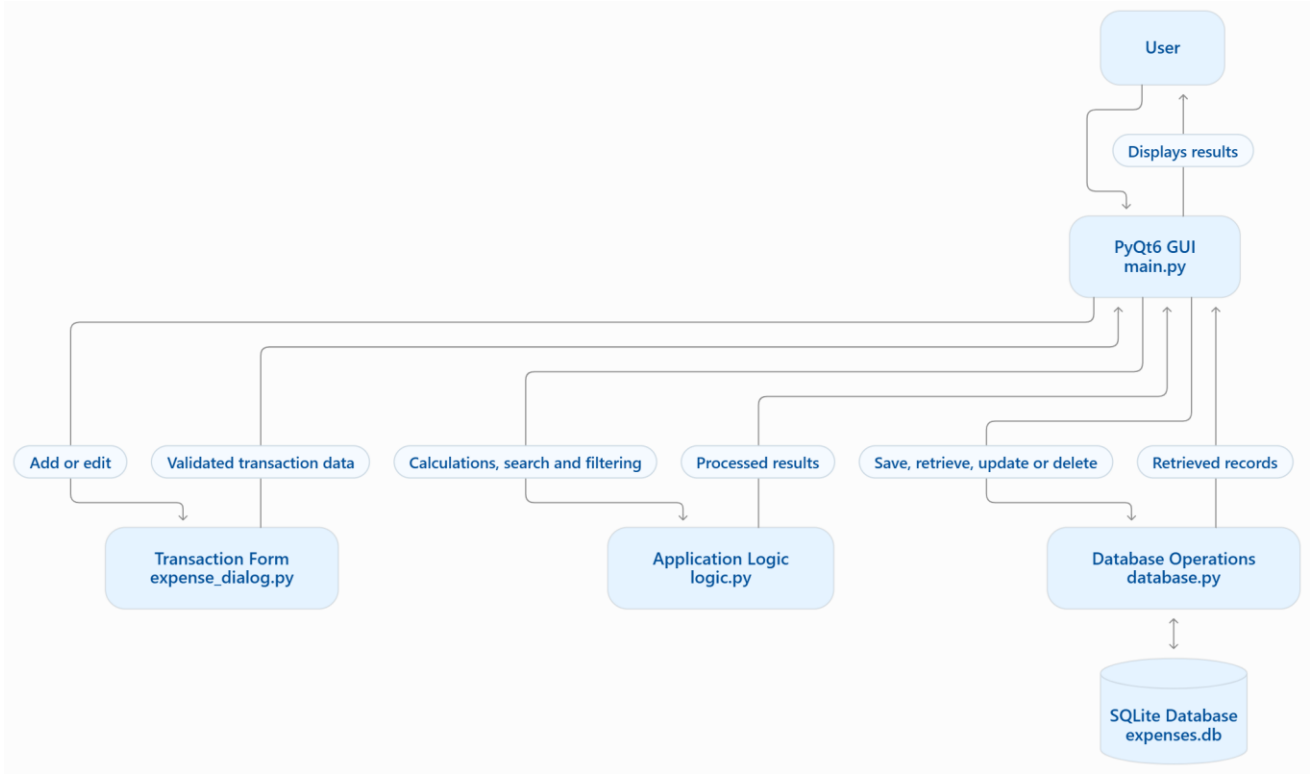


Figure 10: High-level architecture of the Personal Expense Tracker.

5. Implementation

5.1 Development Environment and Tools

Tool / technology	Purpose in the project
Python	Main programming language used to implement the application.
PyQt6	GUI framework used to build the desktop interface.
SQLite	Local relational database used to store application data.
Visual Studio Code	Code editor and development environment.
Windows	Operating system used for development and testing.
Python sqlite3 module	Connects Python to SQLite and executes database queries.
Qt Widgets	Provides the buttons, tables, labels, forms and other interface controls.
Qt Layout Managers	Arrange the widgets within the application window.

5.2 Project Structure

The project is divided into separate Python files to make the code easier to manage. Each file has a different responsibility. The main interface is developed in main.py, while logic.py handles calculations and transaction filtering. The database.py file manages SQLite operations, and expense_dialog.py contains the form used to enter or edit transaction details. This structure keeps the main application code more organized.

File name	Purpose
main.py	Main application file. Creates the window, pages, navigation and connects the different functions.
logic.py	Calculates income, expenses and balance. Also handles searching, month filtering and category summaries.
database.py	Creates the database tables and manages adding, retrieving, updating and deleting records.
expense_dialog.py	Creates the transaction form and validates the information entered by the user.
expenses.db	SQLite database file used to store transactions and budgets.

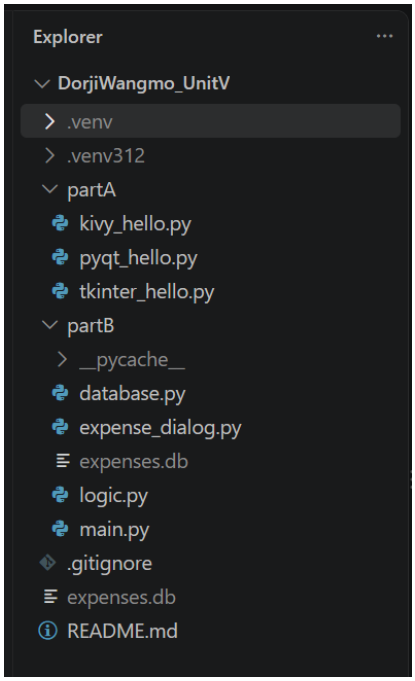


Figure 11: Project folder structure in Visual Studio Code.

5.3 Database Implementation

The database.py file is used to connect the application to the SQLite database. The database file is named expenses.db. The get_connection() function establishes the connection, while create_table() creates the transactions and budgets tables if they do not already exist.

The transactions table stores income and expense records, and the budgets table stores monthly spending limits. The application uses INSERT statements to add new records, SELECT statements to retrieve records, UPDATE statements to edit existing information and DELETE statements to remove records.

After inserting, updating or deleting information, the program uses commit() to save the changes. It then closes the database connection using close(). Parameterised SQL queries are used when passing user information into database operations.

database.py

```
import sqlite3

DATABASE_NAME = "expenses.db"
def get_connection():
    """Create a connection to the SQLite database."""
    return sqlite3.connect(DATABASE_NAME)

def create_table():
    """Create the transactions and budgets tables."""
    connection = get_connection()
    cursor = connection.cursor()

    # Transactions table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS transactions (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            transaction_type TEXT NOT NULL,
            amount REAL NOT NULL,
            category TEXT NOT NULL,
            date TEXT NOT NULL,
            description TEXT
        )
    """)

    # Monthly budgets table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS budgets (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            category TEXT NOT NULL,
            month TEXT NOT NULL,
            amount REAL NOT NULL,
            UNIQUE(category, month)
        )
    """)

    connection.commit()
    connection.close()

def add_transaction(transaction_type, amount, category, date, description):
    """Save a new income or expense transaction."""
    connection = get_connection()
    cursor = connection.cursor()

    cursor.execute("""
        INSERT INTO transactions
        (transaction_type, amount, category, date, description)
        VALUES (?, ?, ?, ?, ?)
    """, (transaction_type, amount, category, date, description))

    connection.commit()
    connection.close()

def get_all_transactions():
    """Retrieve all transactions."""
    connection = get_connection()
    cursor = connection.cursor()

    cursor.execute("SELECT * FROM transactions ORDER BY date DESC")
    records = cursor.fetchall()

    connection.close()
    return records

def update_transaction(transaction_id, transaction_type, amount,
                       category, date, description):
    """Update an existing transaction."""
    connection = get_connection()
    cursor = connection.cursor()

    cursor.execute("""
        UPDATE transactions
```

```
        SET transaction_type = ?, amount = ?, category = ?,
            date = ?, description = ?
        WHERE id = ?
    """, (transaction_type, amount, category, date,
          description, transaction_id))

    connection.commit()
    connection.close()

def delete_transaction(transaction_id):
    """Delete a transaction."""
    connection = get_connection()
    cursor = connection.cursor()

    cursor.execute(
        "DELETE FROM transactions WHERE id = ?",
        (transaction_id,)
    )

    connection.commit()
    connection.close()

# BUDGET FUNCTIONS

def add_budget(category, month, amount):
    """Add a monthly budget for a category."""
    connection = get_connection()
    cursor = connection.cursor()

    cursor.execute("""
        INSERT INTO budgets (category, month, amount)
        VALUES (?, ?, ?)
    """, (category, month, amount))

    connection.commit()
    connection.close()

def get_all_budgets():
    """Retrieve all monthly budgets."""
    connection = get_connection()
    cursor = connection.cursor()

    cursor.execute("SELECT * FROM budgets ORDER BY month DESC")
```

```
    records = cursor.fetchall()

    connection.close()
    return records

def update_budget(budget_id, category, month, amount):
    """Update an existing monthly budget."""
    connection = get_connection()
    cursor = connection.cursor()

    cursor.execute("""
        UPDATE budgets
        SET category = ?, month = ?, amount = ?
        WHERE id = ?
    """, (category, month, amount, budget_id))

    connection.commit()
    connection.close()

def delete_budget(budget_id):
    """Delete a monthly budget."""
    connection = get_connection()
    cursor = connection.cursor()

    cursor.execute(
        "DELETE FROM budgets WHERE id = ?",
        (budget_id,)
    )

    connection.commit()
    connection.close()
```

5.4 Application Logic Implementation

The logic.py file contains functions that process transaction information separately from the graphical interface. It handles financial calculations, searching, month filtering and category-based expense summaries.The calculate_totals() function calculates total income, total expenses and the remaining balance. The search_transactions() function searches records using the description or category.

The filter_by_month() function returns transactions that match a selected month. The calculate_monthly_summary() function uses the month filter to calculate monthly financial information. Finally, calculate_category_summary() calculates the total expenses for each

category.These functions are called from main.py when the user views the dashboard, searches transactions or opens the category summary. Keeping these operations in a separate file makes the code easier to organise and maintain.

logic.py

```
def calculate_totals(transactions):
    """Calculate total income, expenses and balance."""

    total_income = 0
    total_expenses = 0

    for transaction in transactions:
        transaction_type = transaction[1]
        amount = transaction[2]

        if transaction_type == "Income":
            total_income += amount
        elif transaction_type == "Expense":
            total_expenses += amount

    balance = total_income - total_expenses

    return total_income, total_expenses, balance

def search_transactions(transactions, keyword):
    """Search transactions by description or category."""

    keyword = keyword.lower()

    results = []
    for transaction in transactions:
        description = transaction[5] or ""
        category = transaction[3]

        if (keyword in description.lower()
                or keyword in category.lower()):
            results.append(transaction)

    return results

def filter_by_month(transactions, month):
    """Filter transactions by month (YYYY-MM)."""
```

```
    results = []

    for transaction in transactions:
        transaction_date = transaction[4]

        if transaction_date.startswith(month):
            results.append(transaction)

    return results

def calculate_monthly_summary(transactions, month):
    """Calculate income, expenses and balance for a month."""

    monthly_transactions = filter_by_month(transactions, month)

    return calculate_totals(monthly_transactions)

def calculate_category_summary(transactions):
    """Calculate total expenses for each category."""

    category_totals = {}

    for transaction in transactions:
        transaction_type = transaction[1]
        amount = transaction[2]
        category = transaction[3]

        if transaction_type == "Expense":
            if category not in category_totals:
                category_totals[category] = 0

            category_totals[category] += amount

    return category_totals
```

5.5 GUI Implementation

The graphical interface of the Personal Expense Tracker was developed using PyQt6. The main interface is created in main.py using a QMainWindow class. Different widgets and layout managers are used to arrange the application sections.

The application has a sidebar for navigation and a QStackedWidget to hold the five main pages. Users can select Dashboard, Transactions, Reports & Analytics, Budget Planner or About to access the required section. Each page has its own widgets and functions to perform different operations.

5.5.1 Main Window and Navigation

The main window is created using the ExpenseTracker class, which inherits from QMainWindow. The window title is set to Personal Expense Tracker and its initial size is 1150 × 800 pixels.

A sidebar is created using QWidget and QVBoxLayout. It contains five navigation buttons for Dashboard, Transactions, Reports & Analytics, Budget Planner and About. The QStackedWidget is used to hold the different pages. When a navigation button is clicked, its connected function changes the currently displayed page. The set_active_navigation() function is responsible for switching pages and highlighting the selected navigation button.

5.5.2 Dashboard Implementation

The Dashboard displays three main summary values: total income, total expenses and balance. These values are shown using QLabel widgets inside summary cards.

The update_summary() function calls calculate_totals() from logic.py to calculate the overall financial information. The update_monthly_summary() function uses calculate_monthly_summary() to calculate the values for the selected month.

A QDateEdit widget is used as the month selector. When the selected month changes, the month_changed() function refreshes the monthly summary and the transaction filter.

5.5.3 Transaction Management

The Transactions page allows users to add, edit, delete, search and filter financial records. A QTableWidget displays the saved transactions with six columns: ID, Type, Amount, Category, Date and Description.

The Add Transaction button opens ExpenseDialog from expense_dialog.py. When the user saves the form, the get_data() function returns the entered information and add_transaction() from database.py stores it. For editing, the user selects a transaction from the table. The application retrieves the selected record and opens the same dialog with its existing information. The update_transaction() function then saves the changes.

The Delete Selected button asks for confirmation before removing the selected record. After adding, editing or deleting a transaction, load_transactions() retrieves the latest records and refreshes the displayed information. The search field uses search_transactions() from logic.py to find records by description or category. The month filter uses filter_by_month() to display transactions from the selected month.

5.5.4 Reports and Analytics

The Reports & Analytics page provides a monthly overview of the user's financial activity. It displays monthly income, expenses and balance using QLabel widgets.

The page also uses QProgressBar widgets to compare income and expenses. The expense breakdown section calculates the total spending for each category and displays the results using progress bars.

The update_analytics() function performs the calculations using the transaction records and selected month. A QScrollArea is used to make the page scrollable so that the user can access all the report information.

5.5.5 Budget Planner

The Budget Planner allows users to create and manage monthly budgets for different spending categories. A QComboBox is used to select the category, while a QDoubleSpinBox allows the user to enter the budget amount.

The save_budget() function checks whether the user is adding a new budget or updating an existing one. Before adding a new budget, the application checks whether the same category already has a budget for that month. Duplicate records are prevented using the UNIQUE constraint in the database.

The budget table displays the budget limit, amount spent, remaining amount and percentage of budget used. The application also provides buttons to edit and delete selected budgets.

5.6 Annotated Code Snippets

5.6.1 Establishing a Database Connection

```
import sqlite3

DATABASE_NAME = "expenses.db"

def get_connection():
    return sqlite3.connect(DATABASE_NAME)
```

This code imports the sqlite3 module and sets the database filename. The get_connection() function creates a connection to expenses.db so that the application can perform database operations.

5.6.2 Creating Database Tables

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS transactions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        transaction_type TEXT NOT NULL,
        amount REAL NOT NULL,
        category TEXT NOT NULL,
        date TEXT NOT NULL,
        description TEXT
    )
""")
```

This SQL statement creates the transactions table if it does not already exist. The ID is automatically generated for each record. The NOT NULL constraints ensure that the important transaction fields contain values.

5.6.3 Adding a Transaction

```
cursor.execute("""
    INSERT INTO transactions
    (transaction_type, amount, category, date, description)
    VALUES (?, ?, ?, ?, ?)
""", (transaction_type, amount, category, date, description))
```

This code inserts a new transaction into the database. The question marks are placeholders for the values supplied to the query. This allows the program to pass the transaction information separately from the SQL statement.

5.6.4 Editing a Transaction

```
cursor.execute("""
    UPDATE transactions
    SET transaction_type = ?, amount = ?, category = ?,
        date = ?, description = ?
    WHERE id = ?
""", (transaction_type, amount, category, date,
      description, transaction_id))
```

This code updates an existing transaction. The WHERE id = ? condition identifies the record that needs to be changed, so the application updates the selected transaction instead of all records.

5.6.5 Deleting a Transaction

```
cursor.execute(
    "DELETE FROM transactions WHERE id = ?",
    (transaction_id,)
)
```

This code deletes a transaction using its ID. The application passes the selected transaction ID into the SQL statement to identify which record should be removed.

5.6.6 Creating the Main Application Window

```
class ExpenseTracker(QMainWindow):

    def __init__(self):
        super().__init__()

        self.setWindowTitle("Personal Expense Tracker")
        self.setGeometry(100, 100, 1150, 800)

        database.create_table()
```

The ExpenseTracker class creates the main application window using QMainWindow. The window title and size are set in the constructor. The create_table() function is also called to make sure the required database tables are available when the application starts.

5.6.7 Connecting Buttons to Functions

```
self.add_button.clicked.connect(self.add_transaction)
self.edit_button.clicked.connect(self.edit_transaction)
self.delete_button.clicked.connect(self.delete_transaction)
```

These statements connect the buttons to their respective functions. When a user clicks Add, Edit or Delete, the connected function is called to perform the required operation.

5.6.8 Organising Widgets with Layouts

```
main_layout = QHBoxLayout(central_widget)

sidebar_layout = QVBoxLayout(sidebar_widget)

self.pages = QStackedWidget()

main_layout.addWidget(sidebar_widget)
main_layout.addWidget(self.pages)
```

The QHBoxLayout arranges the sidebar and main page area horizontally. The QVBoxLayout arranges the navigation buttons vertically. The QStackedWidget holds the different application pages and displays the selected page.

5.6.9 Input Validation

```
if amount <= 0:
    QMessageBox.warning(
        self, "Invalid Amount",
        "Please enter an amount greater than zero."
    )
    return

if self.category_input.currentIndex() == 0:
    QMessageBox.warning(
        self, "Missing Category",
        "Please select a transaction category."
    )
    return
```

This code checks whether the entered transaction amount is greater than zero and whether a category has been selected. If the information is invalid, a warning message is displayed and the form is not submitted.

5.6.10 Calculating Financial Summaries

```
balance = total_income - total_expenses

return total_income, total_expenses, balance
```

This code calculates the remaining balance by subtracting total expenses from total income. The function returns the income, expenses and balance values so that the main application can display them on the Dashboard.

6. Testing

6.1 Testing Objectives

Testing was carried out to check whether the main application functions behaved as expected. The tests covered adding, editing and deleting records, searching and filtering transactions, displaying dashboard information, validating input, retaining saved data after reopening the application, and viewing budget and report information.

6.2 Test Cases and Results

No.	Feature	Test procedure	Expected result	Actual result / status
1	Add Expense	Add an expense of Nu. 500 under Food with description Lunch.	The expense should be saved and appear in the transaction records.	Passed
2	Add Income	Add income of Nu.	The income should	Passed

		5,000 under Other with description Freelance Payment.	be saved and appear in the transaction records.	
3	Edit Transaction	Change the income amount to Nu. 6,000 and description to Updated Freelance Payment.	The selected record should show the updated amount and description.	Passed
4	Delete Transaction	Delete the temporary income record.	The selected record should be removed from the displayed records and database.	Passed
5	Search Transaction	Search for the description Lunch.	The matching transaction should be found and displayed.	Passed
6	Month Filtering	Select October 2026 and apply the selected-month filter.	The displayed transactions should be filtered to the selected month.	Passed
7	Dashboard Summary	Open or refresh the dashboard summary.	Income, expenses and balance information should be displayed.	Passed
8	Invalid Transaction Amount	Enter a zero transaction amount.	The application should reject the invalid amount.	Passed
9	Empty Description	Submit a transaction without a description.	The application should reject the empty description.	Passed
10	Data Persistence	Close and reopen the application after saving records.	Previously saved records should still be available.	Passed
11	Invalid Budget Limit	Attempt to enter a zero budget limit	The application should prevent a zero budget from being entered.	Passed
12	Reports & Analytics Scrolling	Scroll through the Reports & Analytics page.	The page should allow the user to access content beyond the visible area.	Passed
13	Budget Planner	Open the Budget Planner page.	The selected month's budget records should be displayed with their budget limits, spending amounts, remaining	Passed

			amounts and usage percentages.	

6.3 Testing Summary

A total of thirteen test cases were recorded for the Personal Expense Tracker. The tests covered adding, editing and deleting transactions, searching, month filtering, dashboard summaries, input validation, data persistence, reports and budget management.

According to the recorded test results, all thirteen test cases passed. These tests helped check whether the main features of the application were working as expected. Testing also helped identify the importance of checking invalid inputs and making sure that saved information was still available after reopening the application.

7. Challenges and Reflection

7.1 Challenges Faced

1. Organizing the GUI

One challenge was arranging the different sections of the application in a clear way. I used a sidebar for navigation and QStackedWidget to switch between the Dashboard, Transactions, Reports & Analytics, Budget Planner and About pages. This helped keep the interface organised.

2. Connecting the Database

Another challenge was connecting the application to SQLite and making sure that the information entered through the interface was saved correctly. I used separate database functions for adding, retrieving, updating and deleting records.

3. Updating Transaction Records

I also needed to make sure that the transaction table displayed the latest information after a record was added, edited or deleted. I used the load_transactions() function to retrieve the updated records and refresh the table.

4. Searching and Filtering

Searching and filtering required additional logic because users needed to find transactions by description or category and view records for a selected month. I created separate functions in logic.py to handle these operations.

5. Input Validation

Another area that needed attention was checking the information entered by the user. I added validation in expense_dialog.py to prevent zero amounts, missing categories and empty descriptions from being submitted.

Working on these parts helped me understand how the GUI, application logic and database need to work together in a desktop application.

7.2 Reflection

Through this assignment, I gained practical experience in developing desktop applications using Python GUI frameworks. Exploring PyQt6, Kivy and Tkinter helped me understand how different frameworks create windows, arrange widgets and respond to user actions.

Developing the Personal Expense Tracker also helped me understand how different parts of an application work together. I learned how to connect a PyQt6 interface with SQLite, use separate functions for calculations and searching, and display the results to users.
I also understood the importance of input validation and testing. A program should not only display information but should also handle incorrect input properly and save information correctly.

Overall, this assignment improved my understanding of GUI development, database operations, event handling and organizing Python code into separate files. It also gave me more confidence in developing a functional desktop application.

7.3 Possible Future Improvements

Although the application includes the main features required for managing personal finances, there are still some improvements that could be added in the future

- Add an optional login system so different users can maintain separate records.
- Allow users to export transaction data or reports to CSV, Excel or PDF.
- Introduce visual charts to show spending trends over time.
- Add recurring transactions for regular income or expenses.
- Provide budget warnings when spending approaches or exceeds a category limit.
- Add a backup and restore function for the local database.
- Provide more detailed filters and financial summaries.

8. Conclusion

The Personal Expense Tracker was developed using Python and PyQt6 to help users manage their personal income and expenses. The application provides different sections for recording transactions, viewing financial summaries, searching and filtering records, and managing monthly budgets. SQLite was used to store transaction and budget information locally. The application also uses separate Python files for the graphical interface, database operations, application logic and transaction forms. This helped organise the code and made the different functions easier to manage.

During this assignment, I also explored PyQt6, Kivy and Tkinter through simple Hello message programs. This helped me understand the differences between the frameworks and how they handle widgets, layouts and user interactions.

Overall, completing this project improved my practical knowledge of Python programming, GUI development, database integration, input validation and software testing. There are still features that

could be added in the future, but the application provides the main functions needed for basic personal expense management.

9. References

Python Software Foundation. (n.d.). Python documentation. https://docs.python.org/3/
Riverbank Computing. (n.d.). PyQt6 documentation. https://www.riverbankcomputing.com/static/Docs/PyQt6/
The Qt Company. (n.d.). Qt documentation. https://doc.qt.io/
Kivy. (n.d.). Kivy documentation. https://kivy.org/doc/stable/
Python Software Foundation. (n.d.). tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
SQLite. (n.d.). SQLite documentation. https://www.sqlite.org/docs.html
Microsoft. (n.d.). Visual Studio Code documentation. https://code.visualstudio.com/docs

10. Project Repository

The project repository is hosted on GitHub at the following address:

https://github.com/Dorji-Wangmo123/PRG304-Unit-V-GUI-Assignment